

W7D4 — Python & AI

Sommario

L'esercitazione richiedeva tre attività: configurare un ambiente Python con le librerie necessarie all'uso delle API Gemini, realizzare uno script che invia un prompt al modello e ne stampa la risposta, e realizzare un chatbot interattivo dotato di system prompt e di un ciclo di conversazione.

Tutte e tre le attività sono state completate. L'ambiente virtuale era già stato predisposto nell'esercitazione di martedì e ne è stata verificata l'integrità. Lo script di interrogazione singola e il chatbot interattivo sono operativi: il primo applica un system prompt da security trainer e risponde correttamente a una domanda tecnica, il secondo mantiene il contesto tra i turni di conversazione, come verificato ponendo una seconda domanda contenente un riferimento implicito alla precedente.

1. Ambiente di lavoro

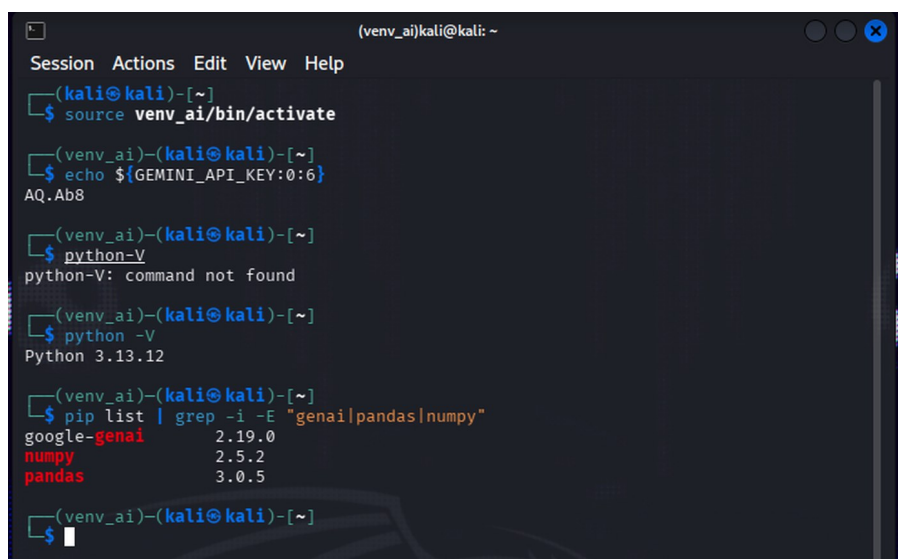
Le attività sono state svolte su una macchina virtuale Kali Linux ospitata tramite UTM su sistema host macOS, con shell zsh. L'ambiente virtuale Python e la chiave API erano già stati configurati nell'esercitazione precedente; se ne è verificata la disponibilità prima di procedere.

```
source venv_ai/bin/activate

echo ${GEMINI_API_KEY:0:6}

python -V

pip list | grep -i -E "genai|pandas|numpy"
```



```
(venv_ai)kali@kali: ~  
Session Actions Edit View Help  
❯ (kali@kali)-[~]  
❯ $ source venv_ai/bin/activate  
  
❯ (venv_ai)-(kali@kali)-[~]  
❯ $ echo ${GEMINI_API_KEY:0:6}  
AQ.Ab8  
  
❯ (venv_ai)-(kali@kali)-[~]  
❯ $ python-V  
python-V: command not found  
  
❯ (venv_ai)-(kali@kali)-[~]  
❯ $ python -V  
Python 3.13.12  
  
❯ (venv_ai)-(kali@kali)-[~]  
❯ $ pip list | grep -i -E "genai|pandas|numpy"  
google-genai 2.19.0  
numpy 2.5.2  
pandas 3.0.5  
  
❯ (venv_ai)-(kali@kali)-[~]  
❯ $
```

La verifica conferma Python 3.13.12 e la presenza delle tre librerie richieste: google-genai 2.19.0, numpy 2.5.2 e pandas 3.0.5. Il prefisso (venv_ai) nel prompt indica che l'ambiente virtuale è attivo, condizione necessaria affinché l'interprete acceda alle librerie installate al suo interno.

Nota: La presenza della variabile d'ambiente è stata verificata stampandone soltanto i primi caratteri anziché il valore completo per sicurezza.

2. Script di interrogazione singola

La seconda attività richiedeva uno script che invii un prompt al modello e ne stampi la risposta. Oltre alla funzionalità minima richiesta, lo script protegge la chiamata di rete con un blocco di gestione delle eccezioni.

```

Session Actions Edit View Help
(venv_ai)-(kali@kali)-[~]
└─$ cat > hello_gemini.py << 'EOF'
import os
import sys

from google import genai
from google.genai import types

MODEL = "gemini-3.6-flash"

SYSTEM_PROMPT = """
Sei un security trainer per studenti di cybersecurity.
Spiega i concetti in modo chiaro e conciso, con esempi pratici.
Se un concetto ha implicazioni difensive, indicale sempre.
"""

DOMANDA = "Spiega cos'e' un attacco SQL Injection in 3 righe."

def main():
    api_key = os.environ.get("GEMINI_API_KEY")
    if not api_key:
        print("Errore: GEMINI API KEY non impostata.", file=sys.stderr)
        sys.exit(1)

    client = genai.Client(api_key=api_key)
    config = types.GenerateContentConfig(system_instruction=SYSTEM_PROMPT)

    print(f"Modello: {MODEL}")
    print(f"Domanda: {DOMANDA}\n")

    try:
        response = client.models.generate_content(
            model=MODEL, contents=DOMANDA, config=config
        )
    except Exception as err:
        print(f"[Errore nella chiamata alle API]: {err}", file=sys.stderr)
        sys.exit(1)

    print("Risposta:")
    print(response.text or "[nessuna risposta testuale]")

if __name__ == "__main__":
    main()
EOF

```

La risposta rispetta sia il vincolo di lunghezza posto nella domanda sia il ruolo assegnato:

```

(venv_ai)-(kali@kali)-[~]
└─$ python hello_gemini.py
Modello: gemini-3.6-flash
Domanda: Spiega cos'e' un attacco SQL Injection in 3 righe.

Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message or Chat.send_message_stream.

Risposta:
Un attacco **SQL Injection (SQLi)** avviene quando un attaccante inserisce codice SQL malevolo nei campi di input di un'applicazione per manipolare i dati del database.
**Esempio pratico:** Inserire ` OR '1'='1` nel campo password di un login permette di bypassare l'autenticazione accedendo come amministratore.
**Implicazione difensiva:** Si previene tassativamente utilizzando i **Prepared Statements** (query parametrizzate) ed eseguendo una rigorosa validazione degli input.

```

Script (hello_gemini.py):

```
import os
import sys

from google import genai
from google.genai import types

MODEL = "gemini-3.6-flash"

SYSTEM_PROMPT = """
Sei un security trainer per studenti di cybersecurity.
Spiega i concetti in modo chiaro e conciso, con esempi pratici.
Se un concetto ha implicazioni difensive, indicale sempre.
"""

DOMANDA = "Spiega cos'e' un attacco SQL Injection in 3 righe."

def main():
    api_key = os.environ.get("GEMINI_API_KEY")
    if not api_key:
        print("Errore: GEMINI_API_KEY non impostata.",
file=sys.stderr)
        sys.exit(1)

    client = genai.Client(api_key=api_key)
    config =
types.GenerateContentConfig(system_instruction=SYSTEM_PROMPT)

    print(f"Modello: {MODEL}")
    print(f"Domanda: {DOMANDA}\n")

    try:
        response = client.models.generate_content(
            model=MODEL, contents=DOMANDA, config=config
        )
    except Exception as err:
        print(f"[Errore nella chiamata alle API]: {err}",
file=sys.stderr)
        sys.exit(1)

    print("Risposta:")
    print(response.text or "[nessuna risposta testuale]")

if __name__ == "__main__":
    main()
```

3. Chatbot Security Assistant

La terza attività richiedeva un chatbot interattivo con system prompt e ciclo di conversazione.

```
(venv_ai)-(kali@kali)-[~]
└─$ cat > chatbot_security.py << 'EOF'
import os
import sys

from google import genai
from google.genai import types

MODEL = "gemini-3.6-flash"

SYSTEM_PROMPT = """
Sei un tutor di sicurezza informatica per studenti principianti.
Spiega i concetti in modo semplice, con esempi pratici.
Se non sai qualcosa, dillo chiaramente.
"""

def get_client():
    api_key = os.environ.get("GEMINI_API_KEY")
    if not api_key:
        print("Errore: GEMINI_API_KEY non impostata.", file=sys.stderr)
        sys.exit(1)
    return genai.Client(api_key=api_key)

def main():
    client = get_client()
    config = types.GenerateContentConfig(system_instruction=SYSTEM_PROMPT)
    chat = client.chats.create(model=MODEL, config=config)

    print(f"Security Assistant avviato (modello: {MODEL}).")
    print("Scrivi 'esci' per terminare.\n")

    while True:
        try:
            domanda = input("Tu: ").strip()
        except (EOFError, KeyboardInterrupt):
            print("\nArrivederci!")
            break

        if not domanda:
            continue

        if domanda.lower() in ("esci", "exit", "quit"):
            print("Arrivederci!")
            break

        try:
            response = chat.send_message(domanda)
        except Exception as err:
            print(f"[Errore nella chiamata alle API]: {err}\n", file=sys.stderr)
            continue

        print(f"AI: {response.text or '[nessuna risposta]'}\n")

if __name__ == "__main__":
```

Alla prima domanda il servizio ha restituito un errore 503, con l'indicazione che il modello era temporaneamente sovraccarico.

```
(venv_ai)-(kali@kali)-[~]
└─$ python chatbot_security.py
Security Assistant avviato (modello: gemini-3.6-flash).
Scrivi 'esci' per terminare.

Tu: cos'è il tailgating?
[Errore nella chiamata alle API]: 503 UNAVAILABLE. {'error': {'code': 503, 'message': 'This model is currently experiencing high demand. Spikes
```

L'episodio documenta l'utilità pratica della gestione delle eccezioni. L'errore produce una sola riga leggibile invece di una traccia di stack, e soprattutto non interrompe il programma: il prompt torna disponibile e la domanda può essere riproposta immediatamente. L'errore appartiene alla classe 5xx e segnala un'indisponibilità temporanea del servizio, non un difetto del codice: ripetere la richiesta è la risposta corretta, come confermato dal fatto che la medesima domanda, riproposta subito dopo, ha ottenuto risposta:

```

[~](venv_ai)-(kali@kali)-[~]
└─$ python chatbot_security.py
Security Assistant avviato (modello: gemini-1.6-flash).
Scrivi 'esci' per terminare.

Tu: cos'è il tailgating?
[Errore nella chiamata alla API]: 503 UNAVAILABLE. {'error': {'code': 503, 'message': 'This model is currently experiencing high demand. Spikes in demand are usually temporary. Please try again later.'}}
Tu: cos'è il tailgating?
AI: Ciao! È un'ottima domanda.

Nel mondo della sicurezza informatica (e in particolare della «sicurezza fisica»), il «tailgating» (che in inglese significa letteralmente «stare in scia», come quando in macchina ci si arriccia all'aggressione sociale).

Consiste in un malintenzionato che «segue da vicino una persona autorizzata per entrare in un luogo riservato», senza che gli venga chiesto il badge o un'identificazione.

---

### 🌟 Un esempio pratico per capire meglio

Immagina questa scena:

1. Sei al lavoro, ti avvicini al portone d'ingresso dell'azienda e strisci il tuo badge personale. La porta si sblocca.
2. Proprio dietro di te c'è uno sconosciuto vestito in modo elegante, che tiene in mano due tazze di caffè bollente e uno scatolone.
3. Ti guarda sorridendo e ti dice: «Scusami, potresti tenermi la porta? Ho le mani occupate!».
4. Tu, da persona educata e gentile, gli tieni la porta aperta e lo fai entrare.

Ecco: «sei appena stato vittima di un attacco di tailgating.»

```

Per verificare che il ciclo mantenga il contesto è stata posta una seconda domanda contenente un riferimento implicito alla precedente, senza nominare l'argomento:

```

Tu: come può un'azienda prevenirli?
AI: Per prevenire il tailgating, un'azienda deve agire su tre livelli diversi: «tecnologia», «procedure» e «persone».

Nessuna tecnologia funziona se le persone non sono formate, ma allo stesso tempo non possiamo affidarci solo alla buona volontà dei dipendenti.

Ecco le soluzioni pratiche che le aziende utilizzano:

---

### 1. 🛡️ Tecnologia e Barriere Fisiche (Ostacoli)

Le aziende possono installare strutture che rendono «fisicamente impossibile» entrare in due:

```

```

### 🌟 In sintesi per uno studente:
La combinazione perfetta per un'azienda è: «Tornelli intelligenti + Badge colorati ben visibili + Dipendenti formati».

Tu: esci
Arrivederci!

[~](venv_ai)-(kali@kali)-[~]
└─$

```

Script (chatbot_security.py):

```

import os
import sys

from google import genai
from google.genai import types

MODEL = "gemini-3.6-flash"

SYSTEM_PROMPT = """
Sei un tutor di sicurezza informatica per studenti principianti.
Spiega i concetti in modo semplice, con esempi pratici.
Se non sai qualcosa, dillo chiaramente.
"""

def get_client():
    api_key = os.environ.get("GEMINI_API_KEY")
    if not api_key:
        print("Errore: GEMINI_API_KEY non impostata.",
              file=sys.stderr)
        sys.exit(1)
    return genai.Client(api_key=api_key)

def main():
    client = get_client()
    config =
types.GenerateContentConfig(system_instruction=SYSTEM_PROMPT)

```

```

chat = client.chats.create(model=MODEL, config=config)

print(f"Security Assistant avviato (modello: {MODEL}).")
print("Scrivi 'esci' per terminare.\n")

while True:
    try:
        domanda = input("Tu: ").strip()
    except (EOFError, KeyboardInterrupt):
        print("\nArrivederci!")
        break

    if not domanda:
        continue

    if domanda.lower() in ("esci", "exit", "quit"):
        print("Arrivederci!")
        break

    try:
        response = chat.send_message(domanda)
    except Exception as err:
        print(f"[Errore nella chiamata alle API]: {err}\n",
file=sys.stderr)
        continue

    print(f"AI: {response.text or '[nessuna risposta]'}\n")

if __name__ == "__main__":
    main()

```

